Doubly Linked List – Memory Allocation

```
    Node *newNode = (Node *)malloc(sizeof(Node));
```

→ *The size of newNode will be equal to the size of a Node structure and newNode is declared as a pointer to Node structure.*

→ *The size of a pointer (newNode) is typically 8 bytes on most modern 64 – bit systems, regardless of the size of the structure it points to.*

→ *The size of the Node structure itself is determined by the sum of the sizes of its members*:

→ *data(an integer) is typically 4 bytes.*

→ *next(a pointer to another Node) is typically 8 bytes.*

→ *prev(a pointer to previous Node) is typically 8 bytes.*

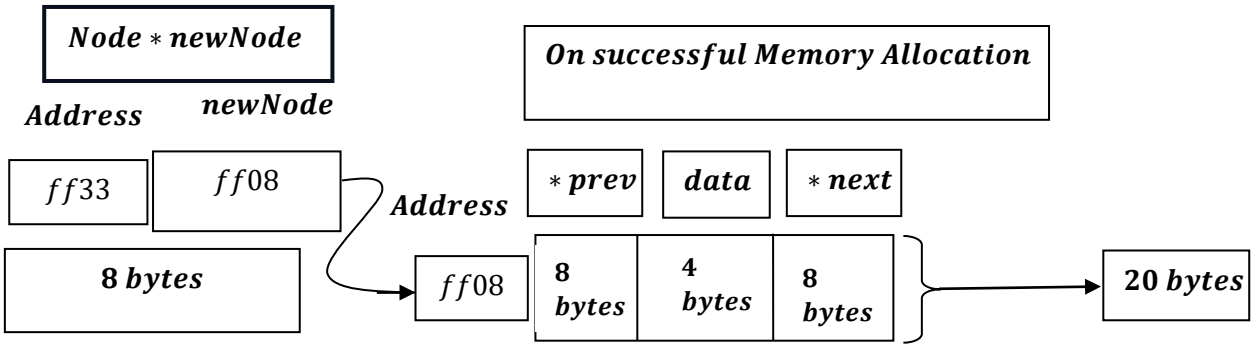
→ *Therefore , the size of the Node structure is typically 20 bytes(4 bytes for data + 8 bytes for next + 8 bytes for prev) .*

```
    Node *newNode = (Node *)malloc(sizeof(Node));
```

What does it mean?

→ *newNode will try to allocate memory for data i.e. 4 bytes , next pointer variable i.e. typically 8 bytes, and prev pointer variable that is typically 8 bytes i.e. total 20 bytes, as discussed earlier , the size of a pointer (newNode) is typically 8 bytes on most modern 64 – bit systems, regardless of the size of the structure it points to.*

Say,



The above is Theoretical , But if we do a reality check:

Theoretical vs. Real − World Allocation

In a real programming environment (like the C + + environments),

the allocation might not be exactly **20 *bytes*** *because of how CPUs access memory*:

- ***Memory Alignment***: *Most* 64 − *bit compilers align data to* **8 − *byte boundaries*** *to improve performance.*

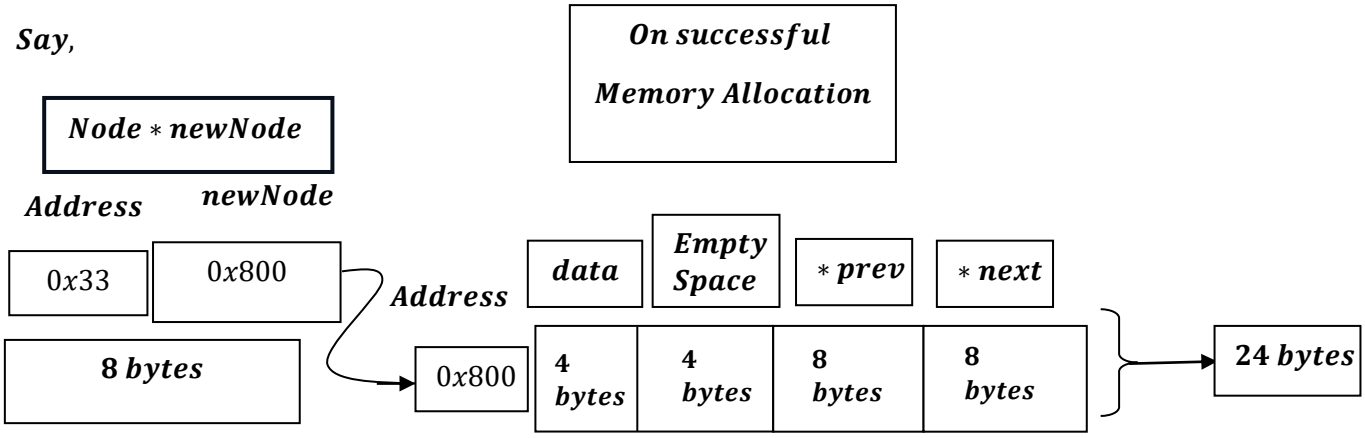
- ***Padding***: *To align the* 8 − *byte next pointer, the compiler will often add* **4 *bytes of*** "***padding***" *after the* 4 − *byte data integer.*

  Additionally, since there is another 8 − *byte pointer (prev), the structure must maintain proper alignment for both pointers.*

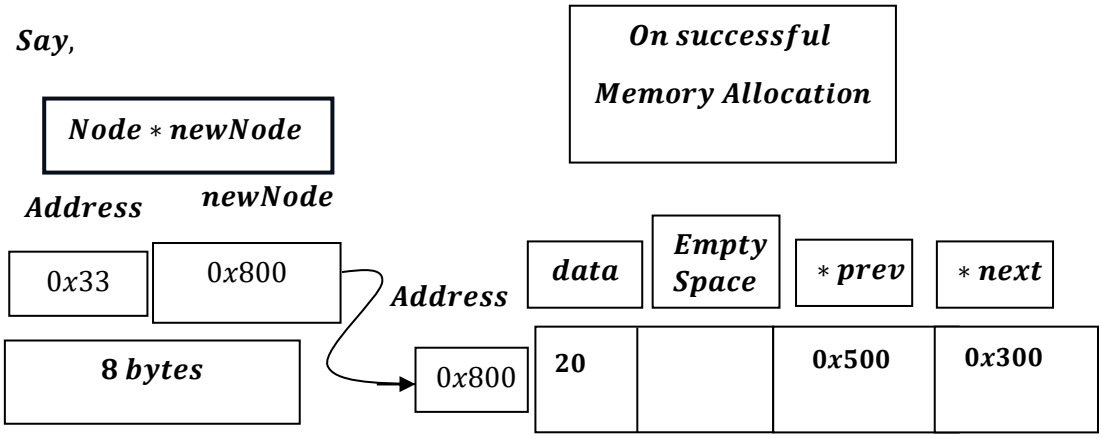
- ***Actual Size***: *Consequently, sizeof(Node) would likely return* **24 *bytes***

  on your system, rather than the theoretical **20 *bytes***.

Offset	Size	Content	Description
0	**4 bytes**	**data**	**Integer value**
4	**4 bytes**	**padding**	**Empty space (alignment)**
8	**8 bytes**	**next**	**Address of next node**
16	**8 bytes**	**prev**	**Address of previous node**



Like:



In a $64-$ *bit environment, the compiler aligns memory to the size of the largest primitive member in a structure* $-$ *in this case, the* $8-$ *byte next pointer.*

Because the data integer only occupies 4 *bytes, the compiler inserts* empty*space* $(padding)$*to ensure the pointer starts at an address that is a multiple of* 8.

Such as:

$0x500 = 5 \times 16^2 + 0 \times 16^1 + 0 \times 16^0 = 5 \times 256 = 1280$

$1280 \div 8 = 160$ $(no\ remainder)$

$0x300 = 3 \times 16^2 + 0 \times 16^1 + 0 \times 16^0 = 3 \times 256 = 768$

$768 \div 8 = 256(no\ remainder)$

Why so ?

As , it depends on $32-$ *bit system and* 64 *system.*

In $64-$ *bit system , memory allocation is total* : 24 *bytes.*

$$\begin{bmatrix} int\ variable - 4\ bytes \\ padding - 4\ bytes \\ Next\ pointer - 8\ bytes \\ prev\ pointer - 8\ bytes \end{bmatrix}$$

Where , as in $32-$ *bit system , memory allocation is total* 12 *bytes.*

Where for next pointer it is allocated $-$ 4 *bytes, prev pointer it is allocated* 4 *bytes and for integer variable , memory allocated* $-$ 4 *bytes.*

Hence for $32-$ *bit system no, such padding is needed.*
